

LocalNotes

RAG Implementation Design

Version <1.0>

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Team Members

Name	GitHub Account
Abdulaziz Ali	Abdulaziz-Ali123
Subat Qutlan	Shaun-Was-Taken
Atharva Patil	a157p624
Malek Kchaou	MK720-dev
Wesley McDougal	wesmcdougal

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Table of Contents

1.	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	References	5
1.5	Overview	5
2.	Architectural Representation	6
2.1	Overview	6
2.2	Component Breakdown	7
3.	Use-Case Diagrams	8
3.1	Component Diagram	8
3.2	User Flow Diagram	8
3.3	Save File Flow Diagram	9
3.4	AI/RAG System Diagram	9
3.5	Quiz System Diagram	10

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Initial Architecture Design

1. Introduction

1.1 Purpose

The purpose of this document is to define the technical architecture and implementation details of a Retrieval-Augmented Generation (RAG) system built using Electron and SQLite with vector search capabilities.

This documentation provides a clear blueprint for implementing a fully local, single-package desktop application that enables users to perform semantic search and AI-assisted querying over notes stored in multiple directories.

1.2 Scope

This document covers:

- Local-first RAG architecture
- Electron application structure
- SQLite database design
- Vector storage and similarity search
- Multi-directory knowledge base isolation
- Indexing and retrieval pipelines
- Packaging considerations

This document does **not** cover:

- Cloud deployment
- Multi-user distributed systems
- Enterprise infrastructure
- Advanced horizontal scaling

The system is designed for single-user desktop environments.

1.3 Definitions, Acronyms, and Abbreviations

This section provides a glossary of terms and acronyms used in the document:

Term	Definition
Electron	A framework that enables building cross-platform desktop applications using web technologies like HTML, CSS, and JavaScript.
Next.js	A React-based framework providing server-side rendering, static site generation, and optimized build processes.
TypeScript	A strongly typed superset of JavaScript that enhances code reliability and scalability.

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Vector Embedding	A high-dimensional numerical representation of text used for semantic similarity comparison.
Vector Search	A similarity-based retrieval method that compares embeddings using distance metrics (e.g., cosine similarity).
RAG	Retrieval Augmented Generation a method of storing content in a way that allows an LLM to be able to access relevant information from the users notes
Embeddings	Vector representation of text used for semantic searching
SQL lite	A lightweight, file-based relational database engine embedded directly within an application.
Directory ID	A unique identifier assigned to a user-selected knowledge base directory to isolate retrieval operations.
Chunking	The process of splitting large documents into smaller segments before embedding.

1.4 References

Electron Documentation

Source: <https://www.electronjs.org>

GitHub

Source: <https://github.com/Abdulaziz-Ali123/LocalNotes>

ReactJs Documentation

Source: <https://react.dev/reference/react>

TailwindCss Documentation

Source: <https://tailwindcss.com/docs/installation/using-vite>

What is RAG and how to implement to whole pipeline locally

Source:  Finally a Local RAG That WORKS!! (+ FULL RAG Pipeline)

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

1.5 Overview

The system is a local-first desktop application that allows users to:

1. Select one or more directories as knowledge bases
2. Index files within those directories
3. Generate embeddings for document chunks
4. Store embeddings in a local SQLite database
5. Retrieve relevant chunks using vector similarity search
6. Inject retrieved context into an LLM prompt

Each directory functions as an isolated knowledge base using a **directory_id** metadata filter to ensure retrieval integrity.

2. Architecture Description

2.1 Overview

The application follows a layered desktop architecture:

User Interface (Renderer Process)



Electron Main Process



RAG Service Layer



SQLite Database (Local File)

The system operates entirely on the user's device, requiring no external database or background service.

Core Architectural Principles

- Local-first design
- Single executable package
- No Docker or external database installation
- Metadata-based directory isolation
- Modular RAG pipeline

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

2.2 Component Breakdown

Component	Description
main/rag/	Handles query embedding, retrieval, context building, and LLM calls
main/indexing/	Handles file change detection, directory traversal, file reading, text splitting, token overlaps, and chunk metadata creation
main/embedding/	Handles calls to embedding model
main/database/	Handles SQL lite connection, document inserts, querying, and vector similarity searches

3. Database Design

3.1 Vector Database Schema Overview

The vector database consists of three primary tables:

1. Directories
2. Files
3. Documents (Vector Embeddings / Chunks)

The design follows a single-collection with metadata filtering approach rather than creating separate tables per directory. This ensures scalability, clean schema management, efficient incremental updates, and flexible cross-directory retrieval.

3.2 Vector Database Schema Overview

3.2.1 Directories Table

The Directories table stores metadata about each user-selected knowledge base directory. When a new directory is opened or created, the user may register it as part of the RAG knowledge database. This enables logical separation while maintaining a unified embedding collection.

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Purpose

- Register knowledge bases
- Assign a unique identifier (directory_id)
- Enable logical data separation
- Support optional cross-directory search

Schema Definition:

```
CREATE TABLE directories (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  path TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Fields:

- id – Unique identifier for the directory (UUID recommended)
- name – User-friendly directory name
- path – Absolute file system path
- created_at – Timestamp of registration

3.2.2 Files Table

The **Files** table tracks individual files within registered directories. This layer enables efficient change detection. By storing a hash of the entire file, the system can determine whether a file needs to be re-chunked and re-embedded.

Purpose

- Detect file-level changes
- Avoid unnecessary reprocessing
- Track file metadata independently from chunks
- Enable clean deletion handling

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Schema Definition

```
CREATE TABLE files (
    id TEXT PRIMARY KEY,
    directory_id TEXT NOT NULL,
    file_path TEXT NOT NULL UNIQUE,
    file_hash TEXT NOT NULL,
    last_modified INTEGER NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(directory_id) REFERENCES directories(id)
);
```

Fields

- **id** – Unique file identifier (UUID recommended)
- **directory_id** – References the parent directory
- **file_path** – Absolute path of the file
- **file_hash** – SHA256 hash of full file contents
- **last_modified** – File system last modified timestamp
- **created_at** – Timestamp of indexing

Rationale

If file_hash remains unchanged during a scan:

- The system skips chunking
- No embeddings are recomputed
- Performance is significantly improved

3.2.3 Chunks (Vector Embeddings) Table

The Chunks table stores all text chunks and their corresponding vector embeddings. Each row represents one chunk of text extracted from a file.

Purpose:

- Store embeddings for semantic retrieval
- Associate each chunk with its originating directory
- Enable filtered or cross-directory vector search

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

Schema Definition:

```
CREATE TABLE chunks (
    id TEXT PRIMARY KEY,
    file_id TEXT NOT NULL,
    content_hash TEXT NOT NULL,
    content TEXT NOT NULL,
    embedding BLOB NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(file_id) REFERENCES files(id),
    FOREIGN KEY(directory_id) REFERENCES directories(id)
);
```

Fields

- **id** – Unique chunk identifier (UUID or deterministic hash recommended)
- **file_id** – References the originating file
- **content_hash** – SHA256 hash of chunk text
- **content** – Text content of the chunk
- **embedding** – Vector embedding stored as binary (Float32Array serialized)
- **created_at** – Timestamp of indexing

3.2.4 Retrieval Logic

The schema enables two retrieval modes:

Single Directory Search

WHERE directory_id = id_of_current_directory

Used when the user queries within a selected knowledge base.

Cross-Directory Search

When the user want to query withing all directories

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

-- No directory filter

Or within multiple selected directories:

WHERE directory_id **IN** (?, ?, ?)

Used when the user enables cross-directory knowledge search.

4. Chunking and Embedding Design

4.1 Chunking Method

Files are split into semantically meaningful chunks using:

- Heading-based segmentation (if markdown)
- Paragraph grouping
- Maximum token cap (recommended: 500–700 tokens)

Each chunk receives a:

`content_hash = SHA256(chunk_text)`

This ensures stable identity even if other parts of the file change.

4.2 Embedding

4.2.1 Overview

The system uses a locally hosted embedding model via llama.cpp to generate vector representations of text chunks.

Specifically, the model used is:

mxlbai-embed-small (F16 precision)

This model is optimized for efficient semantic embedding generation while maintaining low memory usage and fast inference on consumer hardware.

Embeddings are generated locally, ensuring:

- No external API dependency
- Full offline capability
- No data transmitted externally
- Predictable performance

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

4.2.2 Embedding Model Configuration

Model: mxbai-embed-small
Precision: Float16 (F16)
Runtime: llama.cpp

Why mxbai-embed-small

- Lightweight and fast
- Designed specifically for embeddings
- Good semantic quality for academic content
- Suitable for student-level workloads
- Low RAM requirements compared to larger embedding models

This model provides strong retrieval performance without requiring high-end GPUs.

4.2.3 Embedding Generation Workflow

For each new or modified chunk:

1. Receive chunk_text
2. Compute content_hash
3. If hash unchanged → skip
4. If new/modified:
 - Send text to llama.cpp embedding endpoint
 - Receive embedding vector
 - Serialize as binary
 - Store in database

Embedding generation is only triggered when necessary due to file-level and chunk-level hashing.

4.2.4 Query-Time Embedding

At retrieval time:

1. User query is embedded using the same mxbai-embed-small model.
2. Cosine similarity is computed against stored embeddings.
3. Top-K chunks are returned.
4. Retrieved text is passed into the generation model.

Consistency is required:

- The same embedding model must be used for both indexing and querying.

4.2.5 Model Upgrade Considerations

If the embedding model is changed:

- All stored embeddings must be regenerated.
- Stored chunk text allows safe re-embedding.
- File-level and content-level hashing remain unaffected.

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

4.3 Reindexing

4.3.1 Initial Indexing Workflow (Existing Directory)

When a user registers an existing directory into the knowledge database, the system performs a **baseline scan** to establish the initial index state.

Workflow

1. **Scan directory recursively**
 1. Enumerate all supported files (e.g., .md, .txt, extracted PDF text)
2. **For each file:**
 1. Compute file_hash (SHA256 of full file contents)
 2. Check if file exists in files table

Case A: File Not in Database

- Insert new row into files
- Chunk file using semantic chunking strategy
- For each chunk:
 - Compute content_hash
 - Generate embedding
 - Insert into documents
- Store updated file metadata

Case B: File Exists in Database

- Compare stored file_hash with new file_hash

If unchanged:

- Skip file entirely (no re-chunking)

If changed:

1. Re-chunk file
2. Compute content_hash for each chunk
3. Compare against existing chunk hashes for that file

Then:

- Insert new chunks
- Update changed chunks (re-embed)
- Remove stale chunks (no longer present)
- Update file metadata with new file_hash and last_modified

3. Handle Deleted Files

After scanning:

LocalNotes	Version: <1.0>
RAG Implementation Design	Date: 02/14/2026

- Identify files in files table that no longer exist on disk
- Remove:
 - File record
 - All associated chunks in documents

4.3.2 Event-Driven Re-Indexing (Ongoing Updates)

After the initial baseline indexing, the system switches to an event-driven model using a file system watcher. Instead of repeatedly scanning the entire directory, the system reacts to file events.

File Added

1. Compute file_hash
2. Insert into files
3. Chunk and embed
4. Insert chunks into documents

File Modified

1. Compute new file_hash
2. Compare with stored hash
3. If unchanged → ignore event
4. If changed:
 - Re-chunk file
 - Compute content_hash per chunk
 - Insert new chunks
 - Update changed chunks
 - Remove stale chunks
 - Update file metadata

Events are debounced to prevent duplicate processing during rapid saves.

File Deleted

1. Remove file from files
2. Delete all associated chunks from documents